

Имитационное моделирование

Лабораторная работа №5. Аппарат сетей Петри

Исаева Зарина Исмайлбековна

2026-09-06



Содержание

- 1 1 Информация
- 2 2 Цель и задачи
- 3 3 Аппарат сетей Петри
- 4 4 Подготовка проекта
- 5 5 Классическая сеть и арбитр
- 6 6 Имитационные эксперименты
- 7 7 Анимация и отчёт по CSV
- 8 8 Параметрическая серия
- 9 9 Проверка и результаты
- 10 10 Выводы



Раздел 1

1 Информация

1.1 Исходные данные

- Дисциплина: **Имитационное моделирование**
- Тема: **сети Петри и задача обедающих философов**
- Автор: Исаева Зарина Исмайлбековна
- Студенческий билет: 1132232882
- Среда: Julia 1.11, DrWatson, Literate, CairoMakie



1.2 Содержание

1. Цель и требования
2. Формальная модель сети Петри
3. Классическая схема и арбитр
4. Стохастический и детерминированный эксперименты
5. Анимация и анализ сохранённых CSV
6. Параметрическая серия
7. Проверка и выводы

Раздел 2

2 Цель и задачи



2.1 Цель работы

Построить две сети Петри для задачи обедающих философов и экспериментально проверить, как ограничение арбитра предотвращает глобальную взаимную блокировку.

2.2 Обязательные параметры

Сценарий	Параметры	Результат
Базовый	N=5, T=50	две полные траектории
Анимация	N=3, T=30, seed=123	GIF, 5 кадров/с
Анализ CSV	N=5	две панели Eat
Серия	N=3,5,7; 12 повторов	216 траекторий



2.3 Производные форматы

Для каждого из четырёх литературных сценариев созданы:

- чистый Julia-код;
- выполненный Jupyter notebook;
- Quarto QMD;
- автономный HTML-документ.



Раздел 3

3 Аппарат сетей Петри

3.1 Формальное определение

Сеть задаётся позициями P , переходами T , входными и выходными дугами и начальной маркировкой M_0 .

$$M_{k+1} = M_k + (O - I) \cdot j$$

Переход разрешён, когда во всех его входных позициях достаточно фишек.

3.2 Интерпретация позиций

Группа	Смысл
Think _i	философ размышляет
Hungry _i	левая вилка уже захвачена
Eat _i	обе вилки захвачены
Fork _i	вилка свободна
Arbiter	доступно разрешение на захват



3.3 Переходы

1. `TakeLeft_i`: $\text{Think}_i + \text{Fork}_i \rightarrow \text{Hungry}_i$
2. `TakeRight_i`: $\text{Hungry}_i + \text{Fork}_{i+1} \rightarrow \text{Eat}_i$
3. `Release_i`: $\text{Eat}_i \rightarrow \text{Think}_i + \text{Fork}_i + \text{Fork}_{i+1}$

Каждое срабатывание атомарно и сохраняется в журнале событий.

3.4 Критерий deadlock

Deadlock фиксируется напрямую:

$$\{t_j : M \geq I_{.j}\} = \emptyset.$$

Нулевое значение всех Eat_i без этой проверки недостаточно: оно может означать обычную паузу между приёмами пищи.

Раздел 4

4 Подготовка проекта

4.1 Структура

```
/workspace/labs/lab05/project
julia version 1.11.9
scripts/dining_philosophers.jl
scripts/dining_philosophers/dining_philosophers.jl
scripts/dining_philosophers__param.jl
scripts/dining_philosophers__param/dining_philosophers__param.jl
scripts/dining_philosophers_animation.jl
scripts/dining_philosophers_animation/dining_philosophers_animation.jl
scripts/dining_philosophers_report.jl
scripts/dining_philosophers_report/dining_philosophers_report.jl
scripts/setup_environment.jl
scripts/tangle.jl
src/DiningPhilosophers.jl
src/PetriDiningLab.jl
test/runtests.jl
rhktrlwmd@Mac %
```


4.2 Зависимости

```
[deps]
CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
CairoMakie = "13f3f980-e62b-5c42-98c6-ff1f3baf88f0"
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
DrWatson = "634d3b9d-ee7a-5ddf-bec9-22491ea816e1"
IJulia = "7073ff75-c697-5162-941a-fcdaad2a7d2a"
Literate = "98b081ad-f1c9-55d3-8b20-4c87d4299306"
Random = "9a3f8284-a2c9-5f02-9a11-845980a1fd5c"
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"

[compat]
CairoMakie = "0.15"
DataFrames = "1"
DrWatson = "2"
IJulia = "1"
Literate = "2"
julia = "1.11"

[preferences.Makie]
precompile_workload = false

[preferences.CairoMakie]
precompile_workload = false
rhktrlwmd@Mac %
```

4.3 Общий модуль

```

1 module DiningPhilosophers
2
3 using CairoMakie
4 using DataFrames
5 using Random
6
7 export PetriSystem, SimulationResult, build_classical_network
8 export build_arbiter_network, enabled_transitions, fire!
9 export simulate_ode, simulate_stochastic, detect_deadlock
10 export plot_marking_evolution, philosopher_columns, event_summary
11
12 """A place/transition net with separate input and output arc matrices."""
13 struct PetriSystem
14     input::Matrix{Int}
15     output::Matrix{Int}
16     place_names::Vector{Symbol}
17     transition_names::Vector{Symbol}
18     variant::Symbol
19 end
20
21 struct SimulationResult
22     trajectory::DataFrame
23     events::DataFrame
24     deadlock::Bool
25     meal_counts::Vector{Int}
26 end
27
28 nplaces(net::PetriSystem) = length(net.place_names)
29 ntransitions(net::PetriSystem) = length(net.transition_names)
30 incidence(net::PetriSystem) = net.output - net.input
31
32 function philosopher_columns(group::AbstractString, count::Int)
33     [Symbol("$(group)_$(i)") for i in 1:count]
34 end
35
36 function build_network(count::Int; arbiter::Bool)
37     count >= 2 || throw(ArgumentError("At least two philosophers are required"))
38     places = 4count + (arbiter ? 1 : 0)
39     transitions = 3count
40     input = zeros{Int, places, transitions}
41     output = zeros{Int, places, transitions}
42
43     names = reduce(vcat, [philosopher_columns(group, count)
44         for group in ("Think", "Hungry", "Eat", "Fork")])
45     arbiter && push!(names, :Arbiter)
46     actions = reduce(vcat, [philosopher_columns(group, count)
47         for group in ("TakeLeft", "TakeRight", "Release")])

```

DiningPhilosophers.jl в настоящем VS Code

Раздел 5

5 Классическая сеть и арбитр

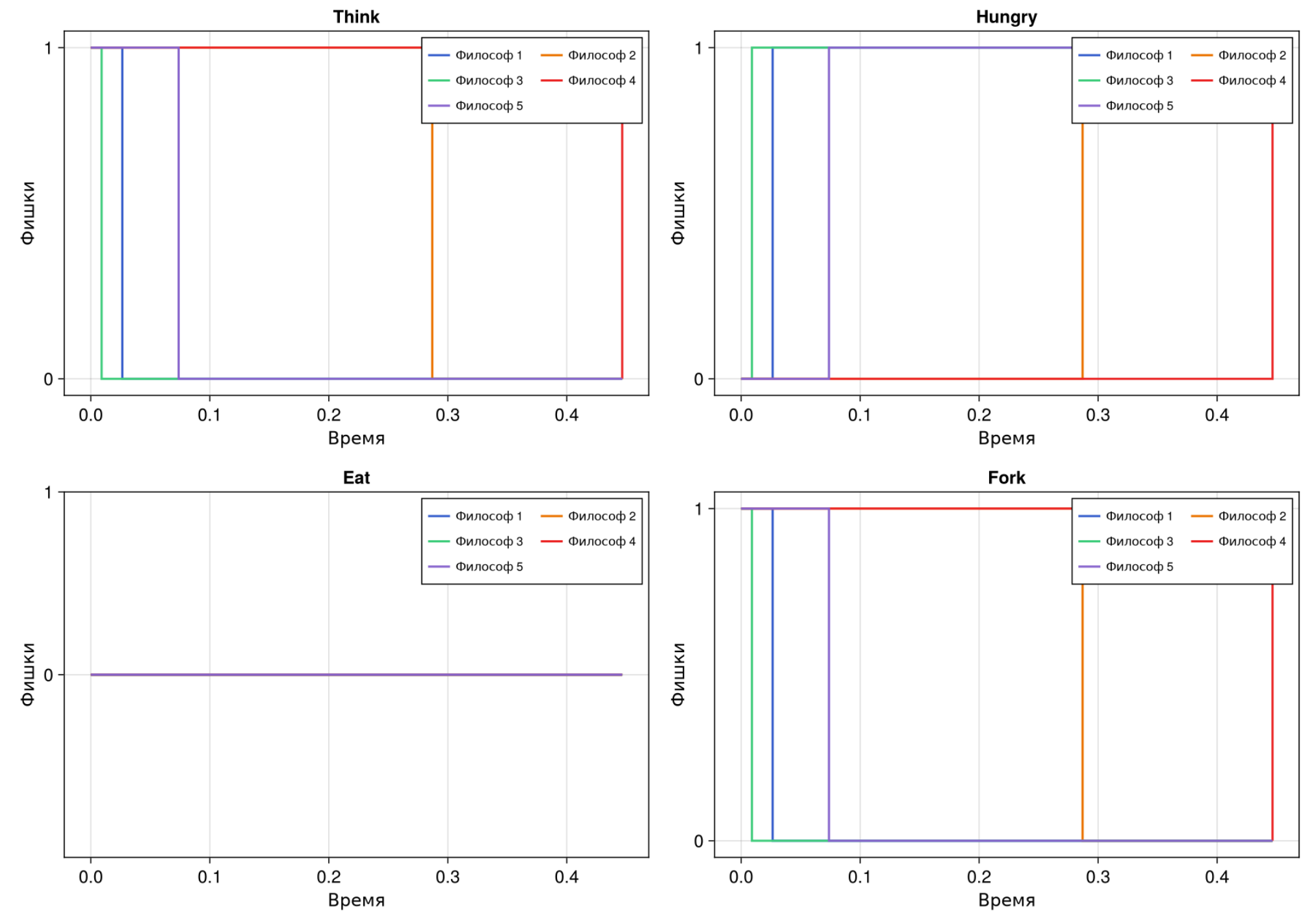
5.1 Классическая схема

Каждый философ сначала захватывает левую вилку. Если все пять переходов `TakeLeft_i` сработали раньше `TakeRight_i`, возникает цикл ожидания:

$$P_1 \rightarrow F_2 \rightarrow P_2 \rightarrow \dots \rightarrow F_1 \rightarrow P_1.$$

5.2 Маркировка классической сети

Классическая сеть: развитие маркировки



Четыре обязательные панели



5.3 Результат классической сети

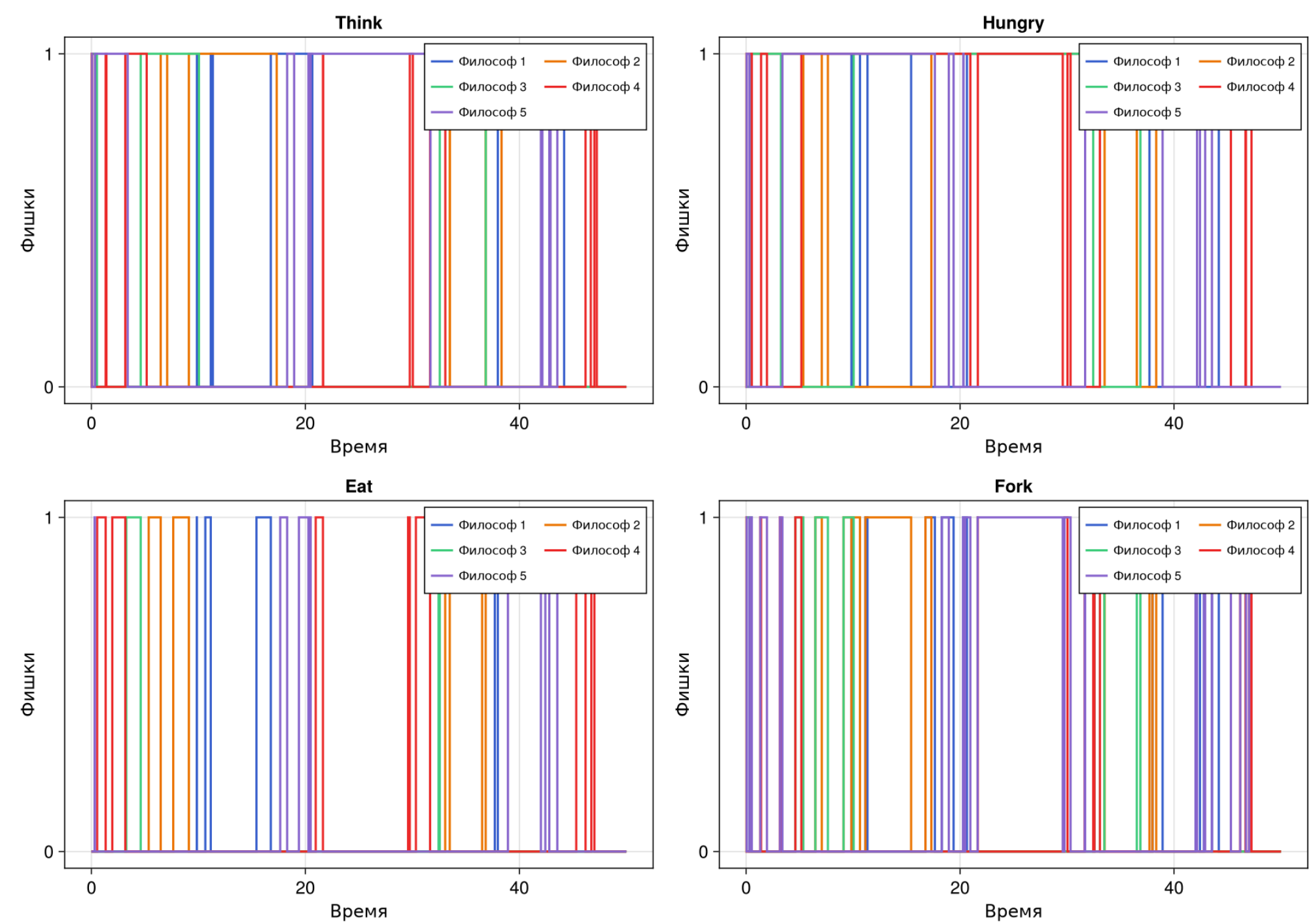
- пять событий;
- время остановки 0,446562;
- завершённых приёмов пищи: 0;
- `deadlock = true`;
- итоговая маркировка: все $Hungry_i=1$.

5.4 Сеть с арбитром

Дополнительная позиция содержит $N-1$ фишек. Один философ всегда остаётся за пределами цикла захвата, поэтому сосед может получить вторую вилку, завершить приём пищи и вернуть ресурсы.

5.5 Маркировка сети с арбитром

Сеть с арбитром: развитие маркировки



Работа сети до горизонта T=50



5.6 Результат арбитра

- 76 событий;
- 24 завершённых приёма пищи;
- индекс Джейна 0,859701;
- конечное время 50,0;
- `deadlock = false`.

Раздел 6

6 Имитационные эксперименты

6.1 Алгоритм Гиллеспи

1. Рассчитать интенсивности разрешённых переходов.
2. Получить $\Delta t = -\ln U/a_0$.
3. Выбрать переход пропорционально интенсивности.
4. Изменить маркировку и записать событие.
5. Остановиться при T или отсутствии переходов.

6.2 Детерминированная аппроксимация

Одна и та же сеть задаёт векторное поле

$$\frac{du}{dt} = (O - I)a(u).$$

Интегрирование выполнено методом RK4 с шагом сохранения 0,1. Проверяется неотрицательность всех непрерывных компонент.

6.3 Выполненный notebook

File Edit View Run Kernel Tabs Settings Help

dining_philosophers.ipynb +

Julia 1.11

Классическая сеть и сеть с арбитром

Базовый опыт сопоставляет две сети Петри для пяти философов. Для каждой сети вычисляется дискретная траектория алгоритмом Гиллеспи и непрерывная mass-action аппроксимация. Все исходные маркировки сохраняются в CSV.

```
[1]: using DrWatson
      @quickactivate "PetriDiningLab"

      ENV["GKSwttype"] = "100"
      using CairoMakie
      using CSV
      using DataFrames
      using Random

      include(sourcedir("DiningPhilosophers.jl"))
      using .DiningPhilosophers
```

Параметры обязательного опыта

```
[2]: const PHILOSOPHERS = 5
      const HORIZON = 50.0
      const BASE_RATES = vcat(fill(2.4, PHILOSOPHERS),
                              fill(0.8, PHILOSOPHERS),
                              fill(1.1, PHILOSOPHERS))

      mkpath(datadir())
      mkpath(plotsdir())
```

Simple 1 Julia 1.11 | Idle Mode: Command Ln 1, Col 1 dining_philosophers.ipynb 1

Параметры базового notebook

6.4 Диагностика в notebook

The screenshot shows a Jupyter Notebook window titled 'dining_philosophers.ipynb'. The interface includes a menu bar (File, Edit, View, Run, Kernel, Tabs, Settings, Help) and a toolbar with icons for file operations and execution. The notebook content is as follows:

```
[2]: "/workspace/labs/lab05/project/plots"
```

Стохастические траектории

```
[3]: classic_net, classic_initial = build_classical_network(PHILOSOPHERS)
      arbiter_net, arbiter_initial = build_arbiter_network(PHILOSOPHERS)

      classic = simulate_stochastic(classic_net, classic_initial, HORIZON;
      rates=BASE_RATES, rng=MersenneTwister(20260501))
      arbiter = simulate_stochastic(arbiter_net, arbiter_initial, HORIZON;
      rates=BASE_RATES, rng=MersenneTwister(20260502))

      CSV.write(datadir("dining_classic.csv"), classic.trajectory)
      CSV.write(datadir("dining_arbiter.csv"), arbiter.trajectory)
      CSV.write(datadir("dining_classic_events.csv"), classic.events)
      CSV.write(datadir("dining_arbiter_events.csv"), arbiter.events)
```

```
[3]: "/workspace/labs/lab05/project/data/dining_arbiter_events.csv"
```

Детерминированная аппроксимация

```
[4]: classic_ode = simulate_ode(classic_net, classic_initial, HORIZON;
      rates=BASE_RATES, saveat=0.1)
      arbiter_ode = simulate_ode(arbiter_net, arbiter_initial, HORIZON;
      rates=BASE_RATES, saveat=0.1)
      CSV.write(datadir("dining_classic_ode.csv"), classic_ode)
      CSV.write(datadir("dining_arbiter_ode.csv"), arbiter_ode)
```

```
[4]: "/workspace/labs/lab05/project/data/dining_arbiter_ode.csv"
```

The bottom status bar indicates 'Simple' mode, '1' cell selected, 'Julia 1.11 | Idle', 'Mode: Command', 'Ln 1, Col 1', and the filename 'dining_philosophers.ipynb'.

Фактический табличный вывод

6.5 Встроенный график notebook

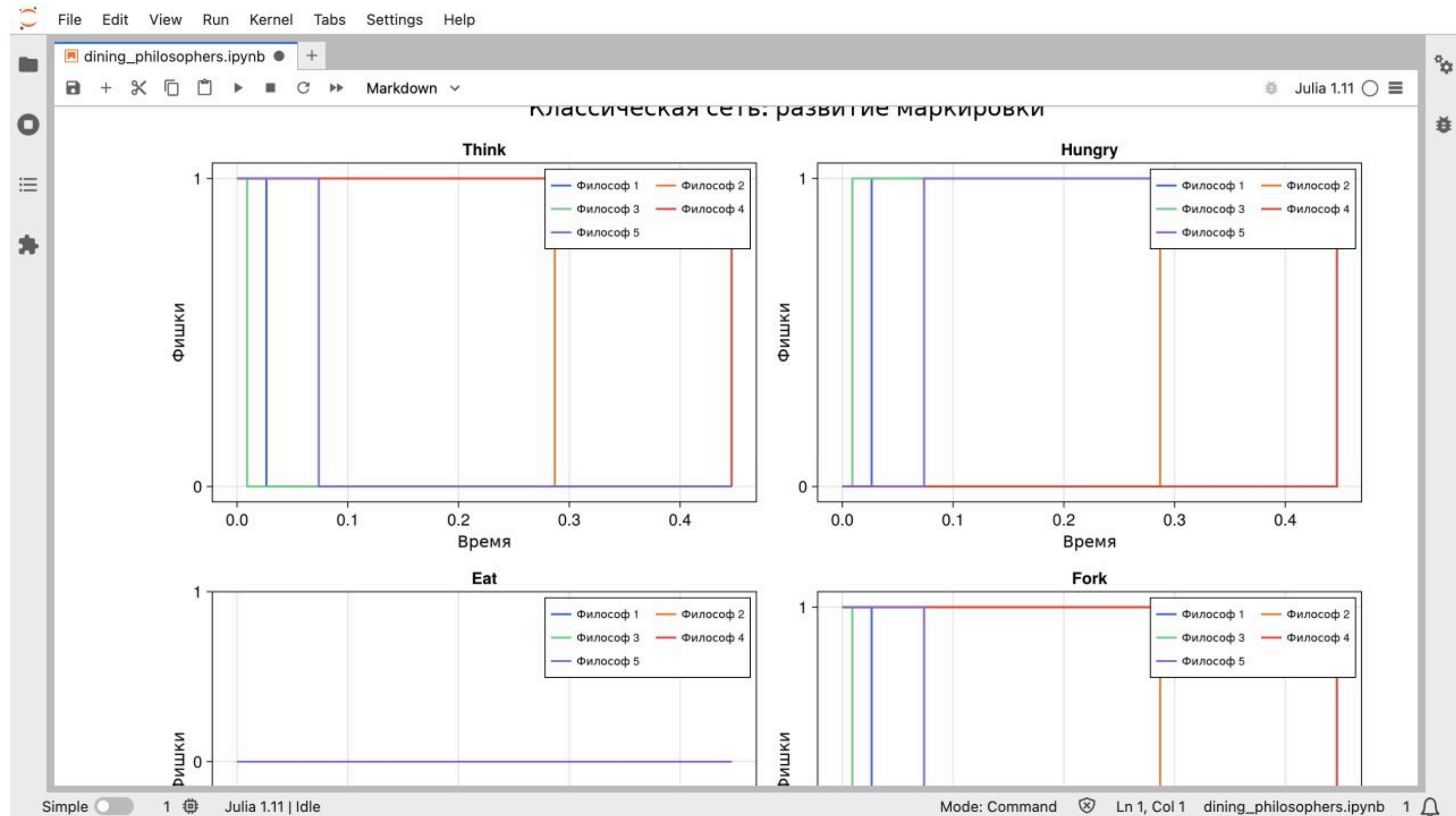
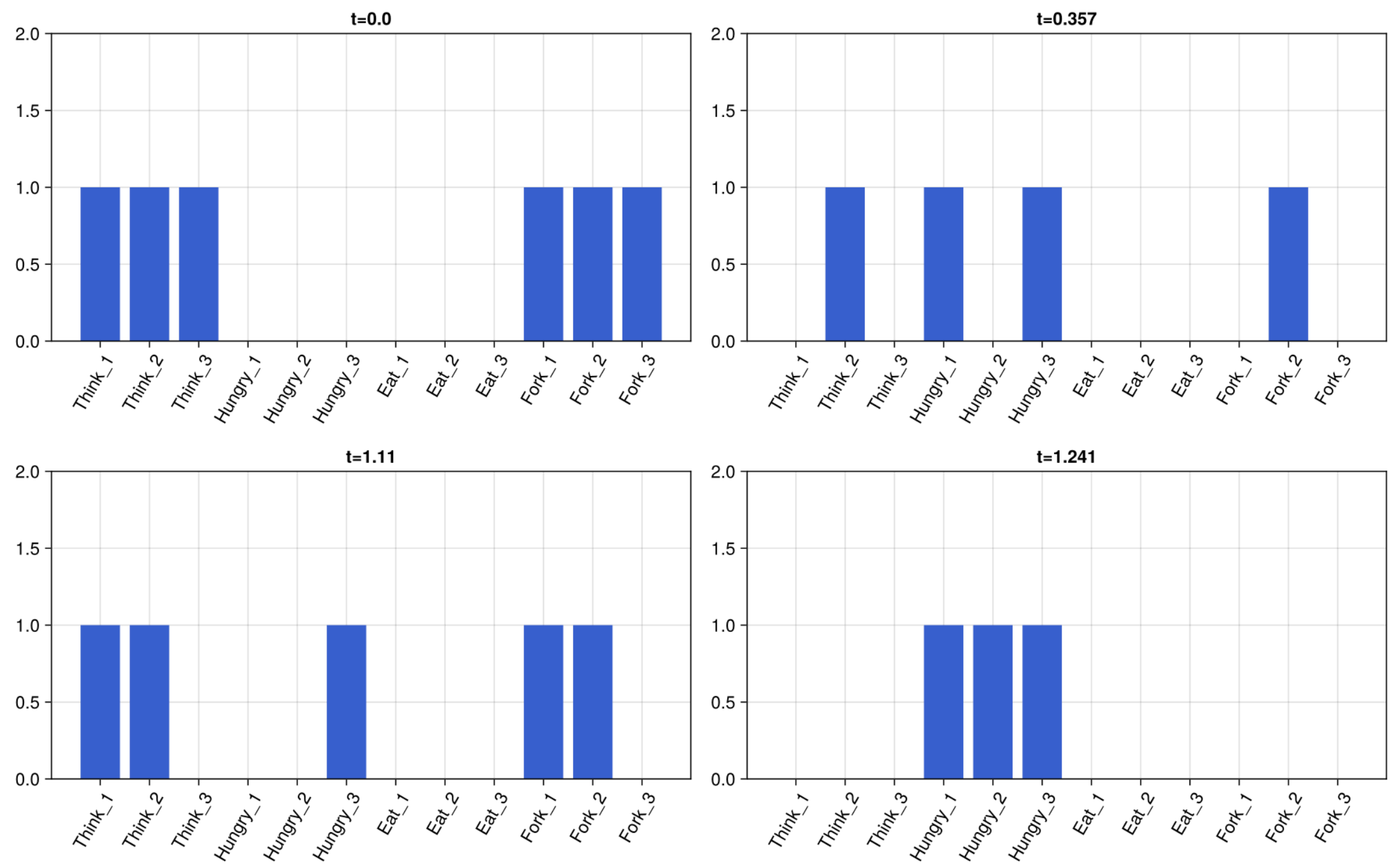


График внутри Jupyter

Раздел 7

7 Анимация и отчёт по CSV

7.1 Анимация N=3



Контрольные маркировки GIF

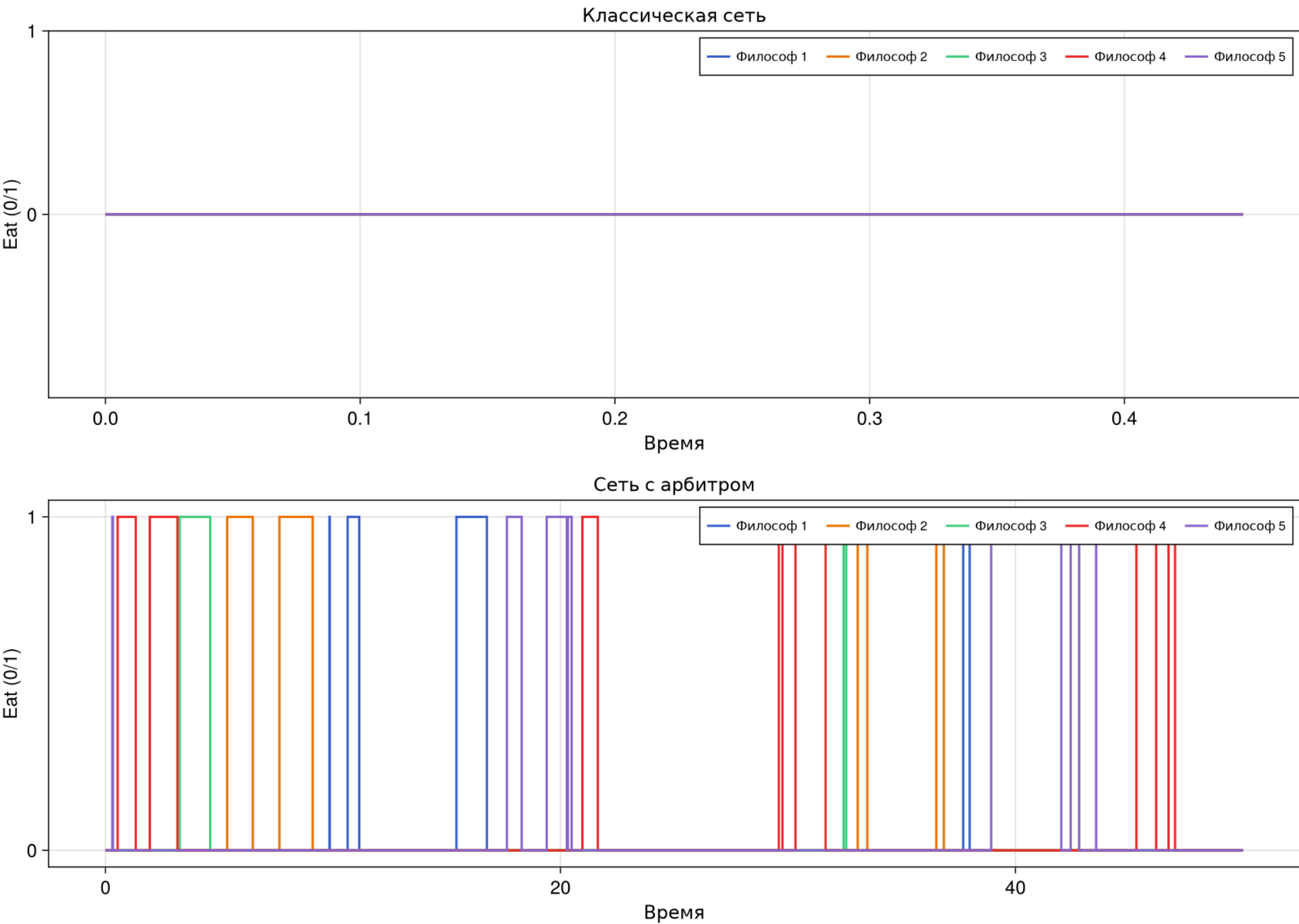
Получено 7 кадров из 6 событий; deadlock наступил в момент 1,240570. Частота GIF — 5 кадров/с согласно текстовому требованию методички.



7.2 Отдельный анализ сохранённых данных

Сценарий `dining_philosophers_report.jl`:

- проверяет наличие двух CSV;
- не запускает модель заново;
- проверяет сортировку времени и столбцы Eat_i ;
- вычисляет активную долю и нулевой хвост;
- сохраняет `final_report.png`.



Классическая сеть и сеть с арбитром



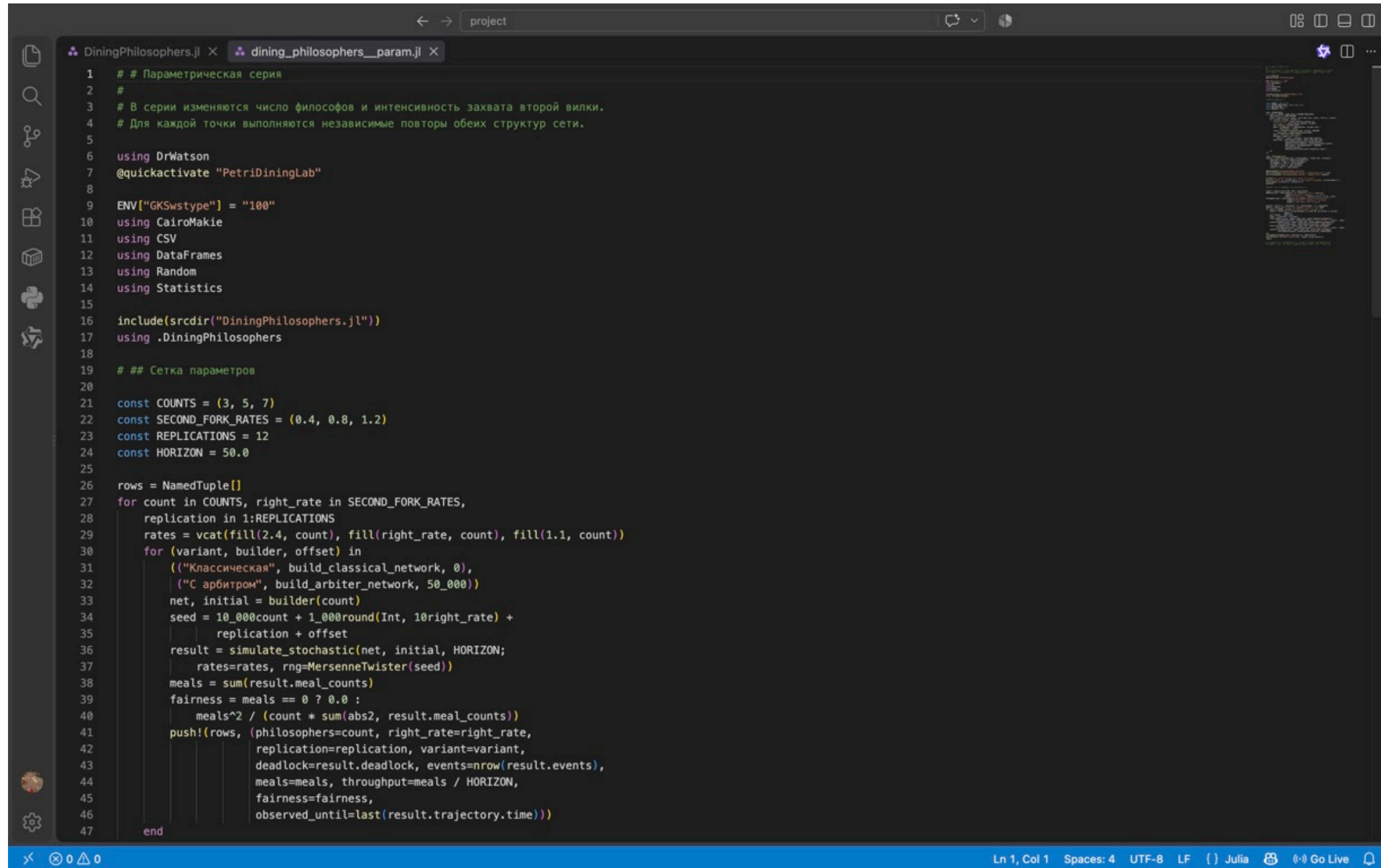
Раздел 8

8 Параметрическая серия

8.1 Дизайн серии

- $N \in \{3, 5, 7\}$;
- скорость второй вилки: 0,4; 0,8; 1,2;
- 12 повторов каждой точки;
- две структуры сети;
- всего 216 независимых траекторий.

8.2 Литературный сценарий



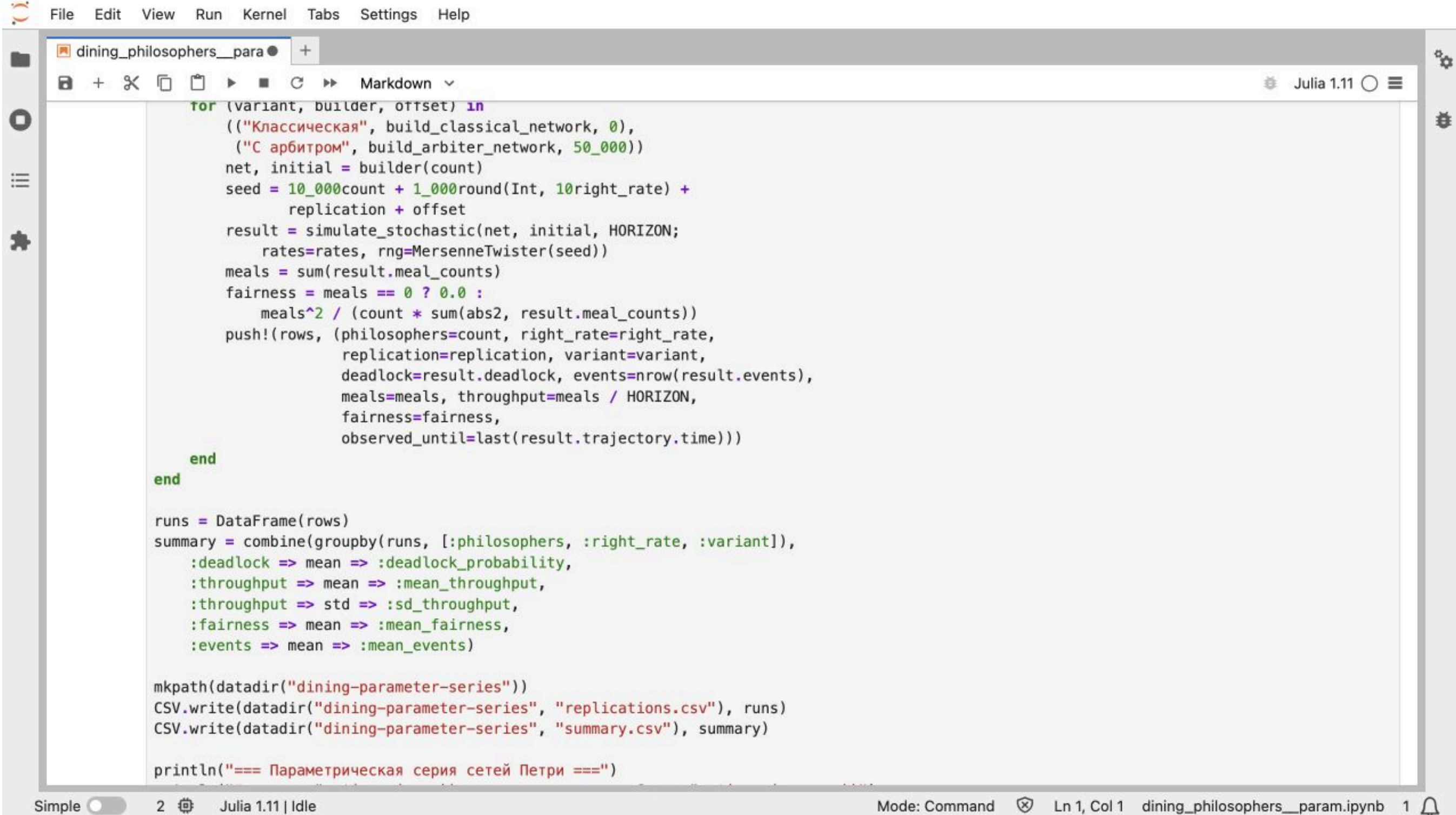
```

1  ## Параметрическая серия
2  #
3  # В серии изменяются число философов и интенсивность захвата второй вилки.
4  # Для каждой точки выполняются независимые повторы обеих структур сети.
5
6  using DrWatson
7  @quickactivate "PetriDiningLab"
8
9  ENV["GKSwttype"] = "100"
10 using CairoMakie
11 using CSV
12 using DataFrames
13 using Random
14 using Statistics
15
16 include(scrdir("DiningPhilosophers.jl"))
17 using .DiningPhilosophers
18
19 ## Сетка параметров
20
21 const COUNTS = (3, 5, 7)
22 const SECOND_FORK_RATES = (0.4, 0.8, 1.2)
23 const REPLICATIONS = 12
24 const HORIZON = 50.0
25
26 rows = NamedTuple{()}
27 for count in COUNTS, right_rate in SECOND_FORK_RATES,
28     replication in 1:REPLICATIONS
29     rates = vcat(fill(2.4, count), fill(right_rate, count), fill(1.1, count))
30     for (variant, builder, offset) in
31         (("Классическая", build_classical_network, 0),
32          ("С арбитром", build_arbiter_network, 50_000))
33         net, initial = builder(count)
34         seed = 10_000count + 1_000round{Int, 10right_rate} +
35             replication + offset
36         result = simulate_stochastic(net, initial, HORIZON;
37             rates=rates, rng=MersenneTwister(seed))
38         meals = sum(result.meal_counts)
39         fairness = meals == 0 ? 0.0 :
40             meals^2 / (count * sum(abs2, result.meal_counts))
41         push!(rows, (philosophers=count, right_rate=right_rate,
42             replication=replication, variant=variant,
43             deadlock=result.deadlock, events=nrow(result.events),
44             meals=meals, throughput=meals / HORIZON,
45             fairness=fairness,
46             observed_until=last(result.trajectory.time)))
47     end

```

Параметрическая серия в VS Code

8.3 Выполненный notebook серии



```

File Edit View Run Kernel Tabs Settings Help
dining_philosophers__para
+
Markdown
Julia 1.11

for (variant, builder, offset) in
  (("Классическая", build_classical_network, 0),
   ("С арбитром", build_arbiter_network, 50_000))
  net, initial = builder(count)
  seed = 10_000count + 1_000round(Int, 10right_rate) +
    replication + offset
  result = simulate_stochastic(net, initial, HORIZON;
    rates=rates, rng=MersenneTwister(seed))
  meals = sum(result.meal_counts)
  fairness = meals == 0 ? 0.0 :
    meals^2 / (count * sum(abs2, result.meal_counts))
  push!(rows, (philosophers=count, right_rate=right_rate,
    replication=replication, variant=variant,
    deadlock=result.deadlock, events=nrow(result.events),
    meals=meals, throughput=meals / HORIZON,
    fairness=fairness,
    observed_until=last(result.trajectory.time)))
end
end

runs = DataFrame(rows)
summary = combine(groupby(runs, [:philosophers, :right_rate, :variant]),
  :deadlock => mean => :deadlock_probability,
  :throughput => mean => :mean_throughput,
  :throughput => std => :sd_throughput,
  :fairness => mean => :mean_fairness,
  :events => mean => :mean_events)

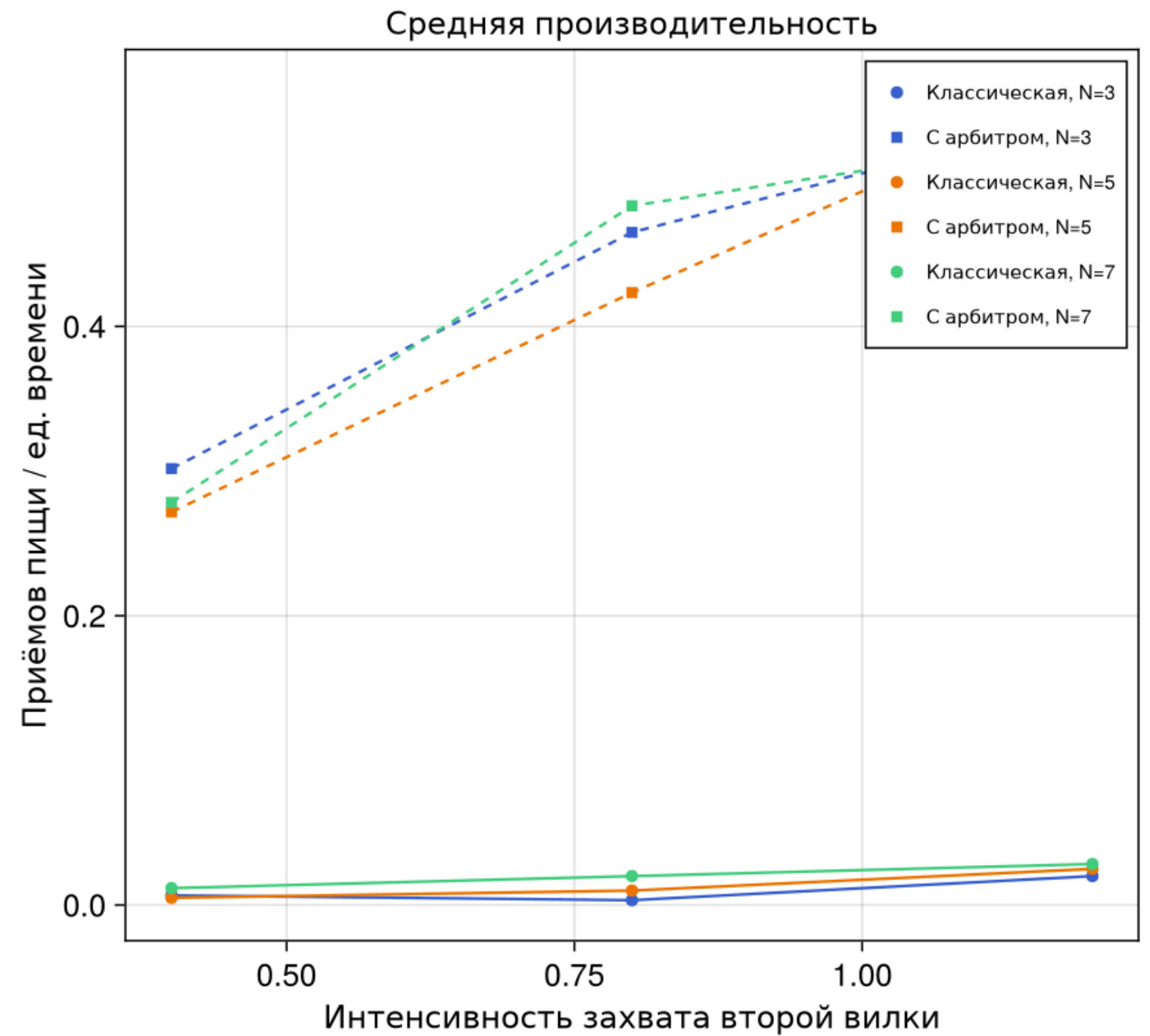
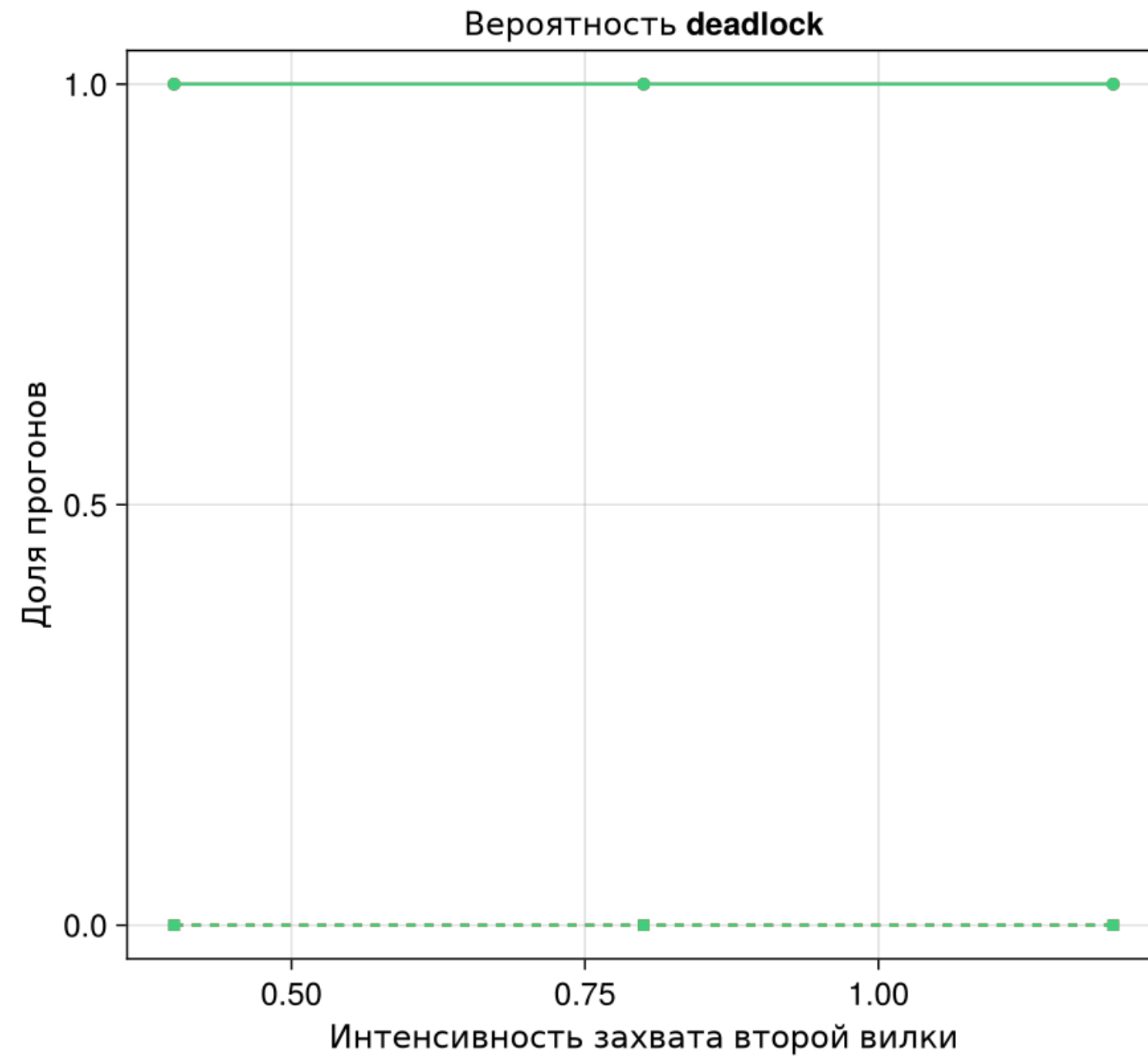
mkpath(datadir("dining-parameter-series"))
CSV.write(datadir("dining-parameter-series", "replications.csv"), runs)
CSV.write(datadir("dining-parameter-series", "summary.csv"), summary)

println("=== Параметрическая серия сетей Петри ===")
Simple 2 Julia 1.11 | Idle Mode: Command Ln 1, Col 1 dining_philosophers__param.ipynb 1

```

Таблица 18 агрегированных комбинаций

8.4 Итог серии



Deadlock и производительность

Классическая сеть: вероятность deadlock 1,0 во всей сетке. Арбитр: 0,0. Для N=5 производительность возрастает с 0,271667 до 0,563333.

Раздел 9

9 Проверка и результаты

9.1 Автоматические тесты

```

Test Summary: | Pass Total Time
Структура классической сети | 4 4 0.5s
Test Summary: | Pass Total Time
Арбитр и инварианты ресурсов | 9 9 0.1s
Test Summary: | Pass Total Time
Известная тупиковая маркировка | 1 1 0.0s
Test Summary: | Pass Total Time
Обязательные параметры и результаты | 4 4 1.7s
Test Summary: | Pass Total Time
Детерминированная аппроксимация | 3 3 0.4s
Test Summary: | Pass Total Time
Производные форматы всех сценариев | 20 20 0.0s
rhktrlwmd@Mac %

```

Шесть наборов, 41 успешная проверка

9.2 Проверенные свойства

- размеры I/O-матриц: 20×15 и 21×15 ;
- начально разрешены пять `TakeLeft_i`;
- $\text{Think}_i + \text{Hungry}_i + \text{Eat}_i = 1$ во всей траектории;
- маркировки неотрицательны;
- классический CSV заканчивается deadlock;
- арбитр достигает $T=50$;
- все четыре notebook выполнены.

9.3 Полученные артефакты

Группа	Количество
самостоятельные сценарии	4
выполненные IPYNB	4
QMD-фрагменты	8
HTML-документы	4
траектории параметрической серии	216
автоматические проверки	41



Раздел 10

10 ВЫВОДЫ



10.1 Основной вывод

Классическая стратегия последовательного захвата ресурсов допускает достижимую тупиковую маркировку. Арбитр с $N-1$ разрешениями устраняет циклическое ожидание и сохраняет активность сети на всём горизонте.



10.2 Роль параметров

Повышение интенсивности захвата второй вилки увеличивает производительность сети с арбитром. В классической структуре изменение скорости не устраняет саму возможность глобальной блокировки.

10.3 Итог работы

- обязательные параметры методички сохранены;
- три именованных сценария и параметрический вариант выполнены отдельно;
- CSV-анализ не повторяет симуляцию;
- код, notebook, QMD, HTML, отчёт и презентация воспроизводимы;
- результаты подтверждены инвариантами и тестами.